

Olist ELT — End-to-End Data Platform

Kafka streaming → MinIO raw lake → PySpark batch → PostgreSQL marts → Streamlit BI

BITS DMML Assignment

Team Member	Student ID
Akshaya Kumar Joish	2025CS05013
Anuj Kumar Srivastava	2025CS05040
Chandan Panda	2025CS05060
Murali Das Mohan	2025CS05052

1. Project Overview

The project builds a production-shaped data platform on the public Olist Brazilian e-commerce dataset. Historical CSVs are replayed as a continuous event stream, ingested through Apache Kafka, landed in a MinIO object-store data lake, transformed by a scheduled PySpark batch job into a PostgreSQL star-schema warehouse, and served through a Streamlit dashboard with BI, deep-dive analytics, and pipeline observability pages. Apache Airflow schedules the batch stage every 20 minutes; the streaming services run continuously as Docker containers. All the components associated with the project are deployed as individual docker images running on a GCP virtual machine. Here are the links to the project repository and the demonstration video.

Code repository: <https://github.com/mdmohan/bits-dmml-assignment/>

Demo Video: <https://drive.google.com/file/d/1itN-jJhvkGFTuWn47SKKnSJEObHsge5i/view?usp=sharing>

2. High-Level Architecture

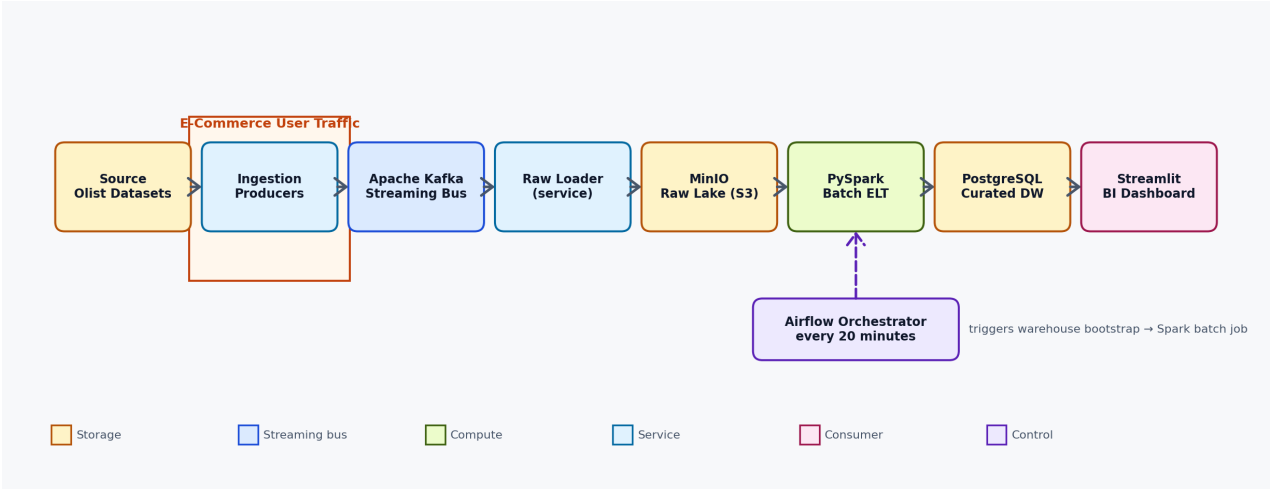


Figure 1. High-level data flow and Airflow control plane.

3. Repository / Code Structure

```
bits-dmml-assignment/
├── ingest/                # Event producers – Kafka producer service
│   ├── producer/         # olist_loader, simulator, event_builder,
│   │                     # dim_sideloader, kafka_producer
│   └── data/             # Olist CSV source datasets
├── lake/                 # Kafka → MinIO raw dumper service
│   └── consumer/         # kafka_consumer, validator, minio_writer, dumper
├── transform/batch-job/  # PySpark ELT batch application
│   ├── processors/       # order, payment, delivery, review, inventory
│   ├── transforms/      # business rules + common_dims
│   ├── spark/           # session, jdbc, upsert_sql, metadata
│   └── utils/           # readers, config
├── warehouse/           # PostgreSQL DW – bootstrap SQL
│   └── bootstrap/       # schema, dim_*, fact_*, indexes, constraints
├── analytics/           # Streamlit BI app – db, queries, app, seed_marts
├── orchestrator/        # Airflow DAG (olist_batch_job_dag.py)
└── data-contracts/      # ingested-data.md, dw-schema.md
```

4. Component Deep-Dive

4.1 Source Data — Olist Datasets

Nine public CSVs covering orders, items, payments, reviews, customers, sellers, products, geolocation, and category translations. These provide the historical truth that the producer replays into a live event stream, preserving real-world entity relationships and timestamp distributions.

4.2 Ingestion Producers (ingest/)

A Python service that loads the Olist CSVs and builds a single timeline ordered by the original event timestamps. A configurable scale factor lets one hour of real traffic replay in seconds. `event_builder.py` wraps every record in a versioned envelope (`event_id`, `event_ts`, `schema_version`, `source_app`). `kafka_producer.py` publishes to the right topic keyed by the business entity so per-entity ordering is preserved. `dim_sideloader.py` publishes dimension snapshots (customers, products, sellers) on their own topics. The service runs continuously in Docker.

4.3 Streaming Bus — Apache Kafka

Kafka is a distributed, append-only commit log that decouples producers from consumers. We use five topics — orders, payments, delivery, reviews, inventory — each partitioned and keyed by entity ID to guarantee in-order delivery per key. Producers write with `acks=all` for durability; consumers commit offsets independently, which lets the raw dumper and Spark batch operate at their own pace. Schema version travels in the envelope, enabling safe contract evolution and full replay.

4.4 Raw Loader (lake/)

A long-running Kafka consumer that validates every message against the envelope contract, enriches it with Kafka metadata (topic, partition, offset, `ingest_ts`), and writes immutable JSON-Lines files to MinIO under `s3://datalake/raw/topic=<t>/dt=YYYY-MM-DD/hour=HH/`. Bad records are routed to a dead-letter bucket. Kafka offsets are committed only after a successful MinIO upload, giving at-least-once delivery.

4.5 Raw Lake — MinIO (S3)

MinIO provides S3-compatible object storage for the immutable raw zone. Partitioning by topic, date, and hour enables cheap incremental reads in the batch ELT. The lake is the single source of truth for replay and back-fills — the warehouse can be rebuilt entirely from it.

4.6 Batch ELT — PySpark (transform/batch-job/)

On every Airflow trigger the Spark application reads only the new raw partitions, deduplicates by event_id, validates schema, derives business columns, and computes both dimension and fact rows. Per-topic processors (order, payment, delivery, review, inventory) reuse a common framework for staging, idempotent upserts, and metrics. JDBC writes use SQL MERGE / INSERT ... ON CONFLICT to upsert into PostgreSQL. Each run prints a METRICS_JSON line (raw_count, dim_written, fact_written, duplicates_skipped) that Airflow captures via XCom.

4.7 Curated Warehouse — PostgreSQL (warehouse/)

A clean star schema bootstrapped from versioned SQL files: dimensions (dim_customer, dim_product, dim_seller, dim_date) and facts (fact_order_events, fact_payments, fact_delivery_events, fact_reviews, fact_inventory) plus staging_events_clean for traceability. Indexes and foreign keys are created idempotently so the bootstrap is safe to re-run.

4.8 Orchestration — Apache Airflow (orchestrator/)

A single DAG, olist_batch_job_dag.py, runs every 20 minutes (* / 20 * * * *). Tasks: start → run_warehouse_bootstrap → run_spark_batch → summarize_metrics → cleanup_airflow_logs → end. DockerOperator runs the bootstrap and Spark images on the shared docker network. Retries with backoff and max_active_runs=1 prevent overlapping runs.

4.9 Serving — Streamlit BI Dashboard (analytics/)

The BI layer is a Streamlit app that connects to PostgreSQL through db.py, runs the SQL in queries.py, and renders interactive Plotly charts in app.py. seed_marts.py is a one-time loader that populates the mart tables directly from the Olist CSVs for offline development.

5. BI Dashboard — What End-Users See

The BI layer is a Streamlit app that directly queries tables and views in PostgreSQL and renders interactive Plotly charts. The dashboard is organised in three sections:

- BI Dashboards — sales, payments, delivery, reviews, seller performance.
- Analytics — deep-dives by category, geography, and individual seller.
- Observability — pipeline health, ingestion freshness, and data-quality metrics.

Key KPIs Shown

- Total orders, total revenue, and average order value.
- On-time delivery percentage and average delivery delay (days).
- Average review score and total counts of sellers and customers.
- Month-over-month revenue growth.

6. Data Quality & Observability

- Envelope contract validation at the dumper; bad records routed to dead-letter bucket.
- Schema version stamped on every event for safe contract evolution and replay.
- Spark run metrics — raw_count, dim_written, fact_written, duplicates_skipped — emitted as METRICS_JSON XCom.
- Streamlit Observability page surfaces ingestion freshness, mart row counts, and last-load timestamps.
- Airflow retries with backoff, max_active_runs=1, and automatic log retention pruning.

7. Technology Stack

Layer	Components
Ingestion	Python 3.11, pandas, confluent-kafka, Docker Compose
Streaming	Apache Kafka (KRaft mode), versioned event envelope
Lake & ELT	MinIO (S3 API), PySpark 3.5, Spark SQL, JDBC sink
Warehouse	PostgreSQL 16 with star schema, indexes, foreign keys
Orchestration	Apache Airflow 2 (DockerOperator, XCom metrics)
BI	Streamlit, Plotly, psycopg2

8. Conclusion

The Olist ELT platform delivers a complete, open-source data pipeline from simulated e-commerce events to refreshed business KPIs in under 30 minutes per cycle. Streaming ingestion, an immutable raw lake, an idempotent Spark ELT, a star-schema warehouse, an interactive Streamlit dashboard, and Airflow orchestration combine to produce a system that is replayable, observable, and easy to extend with new event types or marts.